## Session – 7

**Task-1.**

Create a table called Orders with columns: order_id, user_name, total_amount, and order_date. Insert 5 sample rows with different users and order amounts, including at least one NULL value for total_amount.

**Solution:**

```
CREATE TABLE Orders (
   order_id INT PRIMARY KEY,
   user_name VARCHAR(100),
   total_amount DECIMAL(10,2),
   order_date DATE
);

INSERT INTO Orders (order_id, user_name, total_amount, order_date)
VALUES
(1, 'Rahul', 2500.00, '2026-10-01'),
(2, 'Priya', 1800.00, '2026-10-02'),
(3, 'Rahul', 3200.00, '2026-10-03'),
(4, 'Amit', NULL,    '2026-10-04'),
(5, 'Priya', 4500.00, '2026-10-05');
```

**Task-2.**

Write a SQL query to count how many orders were placed by each user in the Orders table, displaying user_name and the number of orders as order_count.

**Solution:**

```
SELECT user_name, COUNT(*) AS order_count
FROM Orders
GROUP BY user_name;
```

**Task-3.**

Write a SQL query to calculate the average total_amount of all orders in the Orders table, making sure to ignore any NULL values.

**Solution:**

```
SELECT AVG(total_amount) AS average_amount
FROM Orders;
```

**Task-4.**

Suppose you are building a Flipkart-style dashboard: Write a SQL query to find the highest and lowest order amounts (MAX and MIN) from the Orders table, and display both values in a single result row.

**Solution:**

```
SELECT
  MAX(total_amount) AS highest_amount,
  MIN(total_amount) AS lowest_amount
FROM Orders;
```

**Task-5.**

Write a SQL query to calculate the total sales (SUM of total_amount) for all orders, but only include orders where total_amount is not NULL.<br><br>Hint: Use a WHERE clause to filter out NULL values before applying the SUM function.

**Solution:**

```
SELECT SUM(total_amount) AS total_sales
FROM Orders
WHERE total_amount IS NOT NULL;
```